

Guía de Aprendizaje: Estructuras (Structs) en C

Malak Sanchez

Workshop 2026-II

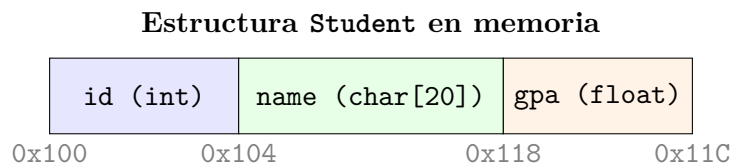
12 de agosto de 2026

1. Motivación

En aplicaciones reales, los datos raras veces existen de forma aislada. Por ejemplo, la información de un estudiante abarca un identificador entero, un nombre (cadena) y un promedio (`float`). Manejar estos datos en variables o arreglos independientes resulta propenso a errores y poco escalable. Las **estructuras** (`struct`) en C nos permiten agrupar variables de distintos tipos bajo un mismo nombre de tipo compuesto.

2. Idea Fundamental

Una estructura es un **tipo de dato definido por el usuario** que agrupa uno o más miembros (campos) en un bloque contiguo de memoria.



3. Sintaxis Básica

Existen dos formas principales de definir e instanciar estructuras en C:

1. Declaración estándar

```
struct Student {  
    int id;  
    float gpa;  
};  
  
struct Student s1; // Requiere la palabra clave 'struct'
```

2. Uso de typedef (Recomendado)

```
typedef struct {  
    int id;  
    float gpa;  
} Student;  
  
Student s1; // Nombre del tipo directo y limpio
```

4. Ejemplo Mínimo Funcional

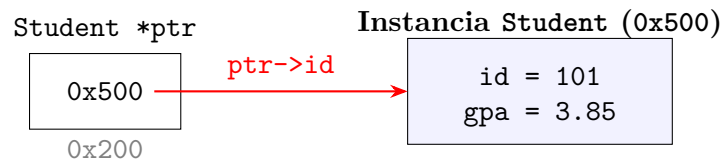
```
1 #include <stdio.h>  
2 #include <string.h>  
3  
4 typedef struct {  
5     int id;  
6     char name[30];  
7     float gpa;  
8 } Student;  
9  
10 int main(void) {  
11     Student s1;  
12     s1.id = 101;  
13     snprintf(s1.name, sizeof(s1.name), "Alice");  
14     s1.gpa = 3.85f;  
15  
16     printf("ID: %d\n", s1.id);  
17     printf("Name: %s\n", s1.name);  
18     printf("GPA: %.2f\n", (double)s1.gpa);  
19  
20     return 0;  
21 }
```

5. Explicación Paso a Paso

1. Se define la estructura `Student` con los miembros `id`, `name` y `gpa`.
2. En la función `main`, se declara una variable local `s1` de tipo `Student`.
3. Se accede a cada campo usando el **operador punto** (`.`).
4. Para asignar cadenas de caracteres a miembros tipo arreglo, se utiliza la función `snprintf` o `strncpy`.

6. Acceso a Miembros: Operador Punto (.) vs Flecha (->)

Cuando se trabaja con estructuras a través de punteros, se utiliza el operador flecha (->) como atajo sintáctico de la dereferenciación.



```

1 Student s1 = {101, "Alice", 3.85f};
2 Student *ptr = &s1;
3
4 // Las siguientes dos expresiones son totalmente equivalentes:
5 (*ptr).gpa = 3.90f; // Dereferenciación explícita + punto
6 ptr->gpa = 3.90f;   // Operador flecha (forma idiomática en C)
  
```

7. Paso de Estructuras a Funciones

Para evitar el costo de copiar bloques de memoria pesados, las estructuras suelen pasarse a las funciones mediante **punteros a constantes** (`const StructType *`) cuando son de solo lectura:

```

1 void print_student(const Student *s) {
2     if (s == NULL) {
3         return;
4     }
5     printf("Student [%d]: %s (GPA: %.2f)\n", s->id, s->name, (
6     double)s->gpa);
7 }
  
```

8. Errores Comunes

- **Omitir el punto y coma final (;):** Olvidar colocar ; al cerrar la definición de la estructura provoca errores de compilación.
- **Asignación directa de cadenas:** Intentar hacer `s1.name = "Alice"`; en lugar de usar `strncpy` o `snprintf`.
- **Confusión entre . y ->:** Intentar usar el operador punto sobre un puntero sin dereferenciarlo previamente.

9. Buenas Prácticas

- Utilizar `typedef` para simplificar la declaración de variables de tipo estructura.
- Pasar estructuras por dirección (`const Student *`) para optimizar el rendimiento y el uso del stack.
- Inicializar los miembros directamente durante la declaración: `Student s = {0};`.

10. Ejercicios

1. Defina una estructura **Rectangle** con miembros **width** y **height** de tipo **double**. Escriba una función que reciba un puntero a **Rectangle** y devuelva su área.
2. Cree una estructura **Point** (x, y) y escriba una función que calcule la distancia euclidiana entre dos puntos.
3. Implemente un programa que lea un arreglo dinámico de N estructuras **Student** y determine el estudiante con el promedio más alto.